

Faggruppemøte om dokumentdatabaser

Hva er NoSQL?

- "Not Only SQL"
- Paraplybegrep for ikke-relasjonelle databaser

Kategorier NoSQL

- **Dokument** – MongoDB
- **Key-Value** – Redis
- **Wide-column** – Cassandra
- **Graf** – Neo4j

Dokumentdatabase

- Lagrer data som selvstendige dokumenter (JSON/BSON)
- Hvert dokument er en komplett enhet
- Fleksibelt schema

Begreper

Relasjonell	Dokument
Tabell	Collection
Rad	Dokument
Kolonne	Felt

<https://www.mongodb.com/docs/manual/reference/sql-comparison/>

Eksempel: ordre normalisert

Tre tabeller koblet med fremmednøkler:

-- orders

id	customer	created_at
1	Ada	2026-05-20

-- order_lines

id	order_id	product_id	qty
1	1	42	2
2	1	17	1

-- products

id	name	price
42	Kaffe	79
17	Krus	149

Eksempel: ordre denormalisert

Hele ordren som ett dokument:

```
{
  _id: 1,
  customer: "Ada",
  createdAt: ISODate("2026-05-20"),
  items: [
    { product: "Kaffe", qty: 2, price: 79 },
    { product: "Krus", qty: 1, price: 149 }
  ]
}
```

Én lesing henter alt – ingen JOIN.

Populære dokumentdatabaser i 2026

- MongoDB
- Couchbase
- RavenDB
- Firebase
- DynamoDB (Amazon)
- Cosmos DB (Microsoft)
- Firestore (Google)

Kort om MongoDB

- Startet **2007**
- Lanserte MongoDB i **2009**
- Lever av **MongoDB Atlas** – managed tjeneste i sky
- Navnet kommer fra **humongous**

Hvordan snakker man med MongoDB?

- Ingen tekstbasert spørrespråk som SQL
- Spørringer er **BSON-dokumenter**
- Du bruker en **offisiell driver** for ditt språk

Eksempel C#

```
using MongoDB.Driver;
using MongoDB.Bson;

var client = new MongoClient("mongodb://localhost:27017");
var db = client.GetDatabase("shop");
var users = db.GetCollection<BsonDocument>("users");

var filter = Builders<BsonDocument>.Filter
    .Eq("address.city", "Oslo");

var result = await users.Find(filter).ToListAsync();
```

Eksempel TypeScript

```
import { MongoClient } from "mongodb";

const client = new MongoClient("mongodb://localhost:27017");
const db = client.db("shop");
const users = db.collection("users");

const result = await users.find({ "address.city": "Oslo" }).toArray();
```

Felles for alle drivere

- Lager et BSON-dokument
- `find`, `insert` osv. er bare hjelpemetoder
- Kunne ha blitt kjørt med `runCommand`

Hva sendes egentlig til serveren?

Hele kommandoen er ett BSON-dokument. Første felt angir operasjonen:

```
// find
{ find: "users", filter: { "address.city": "Oslo" }, $db: "shop" }

// insert
{ insert: "users", documents: [ { name: "Ada" } ], $db: "shop" }

// delete
{ delete: "users",
  deletes: [ { q: { "address.city": "Oslo" }, limit: 0 } ],
  $db: "shop" }
```

CRUD – Insert

```
db.users.insertOne({  
  name: "Ada",  
  email: "ada@example.com",  
  address: { city: "Oslo", zip: "0123" },  
  tags: ["admin", "developer"],  
});
```

CRUD – Find

```
db.users.find();
```

```
db.users.findOne({ name: "Ada" });
```


CRUD – Update

```
db.users.updateOne({ name: "Ada" }, { $set: { "address.city": "Bergen" } });
```

CRUD – Delete

```
db.users.deleteOne({ name: "Ada" });
```

Filter – likhet og sammenligning

```
db.users.find({ "address.city": "Oslo" });
```

```
db.users.find({ age: { $gte: 18, $lt: 65 } });
```

Filter – `$in` og logiske operatorer

```
db.users.find({ status: { $in: ["active", "pending"] } });  
  
db.users.find({  
  $or: [{ tags: "admin" }, { "address.city": "Oslo" }],  
});
```

Nøstede felter med dot-notation

```
db.users.find({ "address.zip": "0123" });
```

Bruk `"felt.subfelt"` i strenger for å gå ned i objekter.

Spørringer mot arrays

```
// Matcher hvis "admin" finnes i tags
db.users.find({ tags: "admin" });

// Må inneholde begge verdier
db.users.find({ tags: { $all: ["admin", "developer"] } });
```

Match på objekt i array – `$elemMatch`

```
db.orders.find({  
  items: {  
    $elemMatch: { product: "kaffe", qty: { $gte: 2 } },  
  },  
});
```

Samme array-element må matche alle betingelsene.

Projection – inkluder felter

```
db.users.find({ "address.city": "Oslo" }, { name: 1, "address.zip": 1 });
```

1 = ta med feltet. Resten utelates.

Projection – fjern felter

```
db.users.find({ "address.city": "Oslo" }, { email: 0, tags: 0 });
```

0 = utelat feltet. Resten tas med.

Hvorfor dokumentdatabase?

- **Datamodellen matcher koden** – dokumenter ligner objekter
- **Fleksibelt schema** – enklere å iterere, færre migreringer
- **Hierarkiske data uten JOIN** – les hele "aggregatet" i ett kall
- **Polymorfiske data** – ulike felter per dokument uten NULL-er
- **Naturlig for JSON-data** – API-er, hendelser, konfigurasjon

Når passer det bra?

- Forenkler applikasjonen sitt "datalag"
- Denormalisering:
 - Forenkler applikasjonen
 - Gir bedre ytelse
 - Når livssyklusen til "under-dataen" styres av objektet (aggregate)
- Det går greit om denormalisert data er feil
 - Kritisk om yndlingsfargen din står som "Gray" eller "Grey"?
- Det gir mening at dataen "kopieres over" slik de var da
 - Ordresystem

Når passer det dårlig?

- Mange relasjoner på kryss og tvers
- Når du ikke får gevinst av denormalisering
- Økt kompleksitet i å holde denormalisert data oppdatert

Normalized data models describe relationships using references between documents. In general, use normalized data models in the following scenarios:

- When embedding would result in duplication of data but would not provide sufficient read performance advantages to outweigh the implications of data duplication.
- To represent more complex many-to-many relationships.
- To model large hierarchical data sets.

Lenker

- <https://www.mongodb.com/resources/products/fundamentals/why-use-mongodb>
- <https://www.mongodb.com/company/blog/mongodb/6-rules-of-thumb-for-mongodb-schema-design> ★
- <https://www.youtube.com/watch?v=b2F-DItXtZs>

Diskusjon

Gode use-cases fra arbeidshverdagen?